

Analysis of the two GenAI Programs

2.

a) The two GenAI programs I used were Claude and Google Gemini.

b) I accessed Claude through the Claude website and Google Gemini through the Gemini website.

c) I used the same prompt for both Claude and Gemini:

Generate C code to implement the following requirements. The program will prioritize emails for a busy company CEO. You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can move toward the front based on priority. You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritization for the CEO. You must create functions from scratch. Do not include pre-existing heap modules. The program should support the commands EMAIL, NEXT, READ, and COUNT. Emails should be prioritized in this order: Boss, Subordinate, Peer, ImportantPerson, and OtherPerson. If two emails have the same sender priority, the newer date should have higher priority. The date format is MM-DD-YYYY.

d)

Claude

```
/* =====
 * email_priority.c
 *
 * Prioritizes a CEO's inbox using a MaxHeap (array/list based,
 * built from scratch -- no library heap module is used).
 *
 * Commands (read one per line from stdin):
 * EMAIL <sender category>,<subject line>,<date MM-DD-YYYY>
 * NEXT
 * READ
 * COUNT
 *
 * Priority (highest to lowest):
 * Boss > Subordinate > Peer > ImportantPerson > OtherPerson
 * Ties (same category) are broken by date: newest date wins.
```

```

* ===== */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX_CATEGORY_LEN 20
#define MAX_SUBJECT_LEN 256
#define MAX_DATE_LEN 11 /* MM-DD-YYYY + '\0' */
#define MAX_LINE_LEN 512
#define INITIAL_CAPACITY 16
/* -----
* Email record stored in the heap
* ----- */
typedef struct {
    char category[MAX_CATEGORY_LEN];
    char subject[MAX_SUBJECT_LEN];
    char date[MAX_DATE_LEN];
    int priority; /* higher number = higher priority category */
    long dateValue; /* YYYYMMDD encoding, higher = newer */
} Email;
/* -----
* MaxHeap: list (dynamic array) based implementation
* ----- */
typedef struct {
    Email *data;
    int size;
    int capacity;
} MaxHeap;
/* ----- Heap lifecycle ----- */
void heapInit(MaxHeap *h, int initialCapacity) {
    h->data = (Email *) malloc(sizeof(Email) * initialCapacity);
    if (h->data == NULL) {
        fprintf(stderr, "Fatal: could not allocate heap memory.\n");
        exit(EXIT_FAILURE);
    }
    h->size = 0;
    h->capacity = initialCapacity;
}
void heapFree(MaxHeap *h) {
    free(h->data);
    h->data = NULL;
    h->size = 0;
    h->capacity = 0;
}

static void heapGrow(MaxHeap *h) {
    int newCapacity = h->capacity * 2;
    Email *newData = (Email *) realloc(h->data, sizeof(Email) * newCapacity);
    if (newData == NULL) {
        fprintf(stderr, "Fatal: could not grow heap memory.\n");
        exit(EXIT_FAILURE);
    }
    h->data = newData;
    h->capacity = newCapacity;
}

```

```

}
/* ----- Comparator -----
 * Returns 1 if email 'a' outranks email 'b' (a belongs closer
 * to the root / front of the queue), 0 otherwise.
 * Rule: higher category priority wins; on a tie, the email with
 * the newer date wins.
 * ----- */
static int isHigherPriority(const Email *a, const Email *b) {
    if (a->priority != b->priority) {
        return a->priority > b->priority;
    }
    return a->dateValue > b->dateValue;
}
static void swapEmails(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}
/* ----- Sift up / down ----- */
static void siftUp(MaxHeap *h, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (isHigherPriority(&h->data[index], &h->data[parent])) {
            swapEmails(&h->data[index], &h->data[parent]);
            index = parent;
        } else {
            break;
        }
    }
}
static void siftDown(MaxHeap *h, int index) {
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < h->size && isHigherPriority(&h->data[left], &h->data[largest])) {
            largest = left;
        }
        if (right < h->size && isHigherPriority(&h->data[right], &h->data[largest])) {
            largest = right;
        }
        if (largest == index) {
            break;
        }
        swapEmails(&h->data[index], &h->data[largest]);
        index = largest;
    }
}
/* ----- Public heap operations ----- */
void heapPush(MaxHeap *h, Email e) {
    if (h->size == h->capacity) {
        heapGrow(h);
    }
}

```

```

    h->data[h->size] = e;
    siftUp(h, h->size);
    h->size++;
}
/* Removes and returns the highest priority email.
 * Caller must ensure the heap is not empty (check heapIsEmpty first). */
Email heapPop(MaxHeap *h) {
    Email top = h->data[0];
    h->size--;
    h->data[0] = h->data[h->size];
    if (h->size > 0) {
        siftDown(h, 0);
    }
    return top;
}
/* Returns a pointer to the highest priority email without removing it.
 * Returns NULL if the heap is empty. */
Email *heapPeek(MaxHeap *h) {
    if (h->size == 0) {
        return NULL;
    }
    return &h->data[0];
}
int heapIsEmpty(MaxHeap *h) {
    return h->size == 0;
}
/* -----
 * Helpers for translating parsed input into heap-ready data
 * ----- */
int getCategoryPriority(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0; /* unknown category: lowest priority */
}
/* Converts MM-DD-YYYY into a comparable long, e.g. 09-17-2026 -> 20260917 */
long dateToValue(const char *date) {
    int month, day, year;
    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3) {
        return 0;
    }
    return (long) year * 10000L + (long) month * 100L + (long) day;
}
/* Trims leading/trailing whitespace in place */
void trim(char *str) {
    /* trim leading */
    int start = 0;
    while (str[start] != '\0' && isspace((unsigned char) str[start])) {
        start++;
    }
    if (start > 0) {
        memmove(str, str + start, strlen(str + start) + 1);
    }
}

```

```

    }
    /* trim trailing */
    int len = (int) strlen(str);
    while (len > 0 && isspace((unsigned char) str[len - 1])) {
        str[len - 1] = '\0';
        len--;
    }
}
/* -----
* Command processing
* ----- */
/* rest points to the text following "EMAIL " on the command line, e.g.
* "Boss,Server outage,09-17-2026"
* Fields are comma delimited. The subject may contain spaces, but not
* commas, per the spec.
*/
void processEmailCommand(MaxHeap *h, char *rest) {
    char category[MAX_CATEGORY_LEN] = {0};
    char subject[MAX_SUBJECT_LEN] = {0};
    char date[MAX_DATE_LEN] = {0};
    char buffer[MAX_LINE_LEN];
    strncpy(buffer, rest, MAX_LINE_LEN - 1);
    buffer[MAX_LINE_LEN - 1] = '\0';
    char *field1 = strtok(buffer, ",");
    char *field2 = strtok(NULL, ",");
    char *field3 = strtok(NULL, ",");
    if (field1 == NULL || field2 == NULL || field3 == NULL) {
        fprintf(stderr, "Malformed EMAIL command, ignoring: %s\n", rest);
        return;
    }
    trim(field1);
    trim(field2);
    trim(field3);
    strncpy(category, field1, MAX_CATEGORY_LEN - 1);
    strncpy(subject, field2, MAX_SUBJECT_LEN - 1);
    strncpy(date, field3, MAX_DATE_LEN - 1);
    Email e;
    strncpy(e.category, category, MAX_CATEGORY_LEN - 1);
    e.category[MAX_CATEGORY_LEN - 1] = '\0';
    strncpy(e.subject, subject, MAX_SUBJECT_LEN - 1);
    e.subject[MAX_SUBJECT_LEN - 1] = '\0';
    strncpy(e.date, date, MAX_DATE_LEN - 1);
    e.date[MAX_DATE_LEN - 1] = '\0';
    e.priority = getCategoryPriority(e.category);
    e.dateValue = dateToValue(e.date);
    heapPush(h, e);
}

void processNextCommand(MaxHeap *h) {
    Email *top = heapPeek(h);
    if (top == NULL) {
        printf("No emails in queue.\n");
        return;
    }
    printf("Sender: %s\n", top->category);
}

```

```

    printf("Subject: %s\n", top->subject);
    printf("Date: %s\n", top->date);
}
void processReadCommand(MaxHeap *h) {
    if (heapIsEmpty(h)) {
        printf("No emails to read.\n");
        return;
    }
    Email e = heapPop(h);
    printf("CEO has read and dealt with: %s (%s)\n", e.subject, e.category);
}
void processCountCommand(MaxHeap *h) {
    printf("Unread email count: %d\n", h->size);
}
/* -----
 * Main: reads commands line by line until EOF
 * ----- */
int main(void) {
    MaxHeap heap;
    heapInit(&heap, INITIAL_CAPACITY);
    char line[MAX_LINE_LEN];
    while (fgets(line, sizeof(line), stdin) != NULL) {
        /* strip trailing newline */
        size_t len = strlen(line);
        while (len > 0 && (line[len - 1] == '\n' || line[len - 1] == '\r')) {
            line[len - 1] = '\0';
            len--;
        }
        if (len == 0) {
            continue; /* skip blank lines */
        }
        if (strncmp(line, "EMAIL ", 6) == 0) {
            processEmailCommand(&heap, line + 6);
        } else if (strcmp(line, "NEXT") == 0) {
            processNextCommand(&heap);
        } else if (strcmp(line, "READ") == 0) {
            processReadCommand(&heap);
        } else if (strcmp(line, "COUNT") == 0) {
            processCountCommand(&heap);
        } else {
            fprintf(stderr, "Unrecognized command: %s\n", line);
        }
    }
    heapFree(&heap);
    return 0;
}

```

Gemini

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_STRING_LEN 256
// Define structure for an Email

```

```

typedef struct {
    char sender[MAX_STRING_LEN];
    char subject[MAX_STRING_LEN];
    char date[11]; // MM-DD-YYYY format
    int priority; // Calculated priority based on sender category
    long date_val; // Numeric value for date comparison (YYYYMMDD)
} Email;
// Define structure for MaxHeap (List-based / Dynamic Array)
typedef struct {
    Email *data;
    int size;
    int capacity;
} MaxHeap;
// Function to map sender categories to priority weights
int get_sender_priority(const char *category) {
    if (strcmp(category, "Boss") == 0) return 5; // Read first
    if (strcmp(category, "Subordinate") == 0) return 4; // Read next
    if (strcmp(category, "Peer") == 0) return 3; // Read next
    if (strcmp(category, "ImportantPerson") == 0) return 2; // Read next
    if (strcmp(category, "OtherPerson") == 0) return 1; // Read last
    return 0;
}
// Function to convert MM-DD-YYYY to a comparable integer YYYYMMDD
long parse_date(const char *date_str) {
    int mm, dd, yyyy;
    sscanf(date_str, "%d-%d-%d", &mm, &dd, &yyyy);
    return (long)yyyy * 10000 + (long)mm * 100 + (long)dd;
}
// Initialize MaxHeap
MaxHeap* create_heap(int capacity) {
    MaxHeap *heap = (MaxHeap *)malloc(sizeof(MaxHeap));
    heap->data = (Email *)malloc(capacity * sizeof(Email));
    heap->size = 0;
    heap->capacity = capacity;
    return heap;
}
// Swap two emails
void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}
// Compare two emails: returns true if email `a` has higher priority than email `b`
int compare_emails(Email a, Email b) {
    if (a.priority != b.priority) {
        return a.priority > b.priority;
    }
    // If priorities are equal, newer email (larger date value) comes first
    return a.date_val > b.date_val;
}
// Heapify up to maintain MaxHeap property after insertion
void heapify_up(MaxHeap *heap, int idx) {
    int parent = (idx - 1) / 2;
    if (idx > 0 && compare_emails(heap->data[idx], heap->data[parent])) {

```

```

        swap(&heap->data[idx], &heap->data[parent]);
        heapify_up(heap, parent);
    }
}
// Heapify down to maintain MaxHeap property after removal
void heapify_down(MaxHeap *heap, int idx) {
    int largest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;
    if (left < heap->size && compare_emails(heap->data[left], heap->data[largest])) {
        largest = left;
    }
    if (right < heap->size && compare_emails(heap->data[right], heap->data[largest])) {
        largest = right;
    }
    if (largest != idx) {
        swap(&heap->data[idx], &heap->data[largest]);
        heapify_down(heap, largest);
    }
}
// Insert email into MaxHeap (EMAIL command implementation)
void insert_email(MaxHeap *heap, const char *sender, const char *subject, const char *date) {
    if (heap->size >= heap->capacity) {
        heap->capacity *= 2;
        heap->data = (Email *)realloc(heap->data, heap->capacity * sizeof(Email));
    }
    Email new_email;
    strcpy(new_email.sender, sender);
    strcpy(new_email.subject, subject);
    strcpy(new_email.date, date);
    new_email.priority = get_sender_priority(sender);
    new_email.date_val = parse_date(date);
    heap->data[heap->size] = new_email;
    heapify_up(heap, heap->size);
    heap->size++;
}
// Display the next email for the CEO (NEXT command implementation)
void peek_next(MaxHeap *heap) {
    if (heap->size == 0) {
        printf("Inbox is empty.\n");
        return;
    }
    Email top = heap->data[0];
    printf("Sender: %s\n", top.sender);
    printf("Subject: %s\n", top.subject);
    printf("Date: %s\n", top.date);
}
// Remove the highest priority email once read (READ command implementation)
void read_email(MaxHeap *heap) {
    if (heap->size == 0) {
        printf("No emails to read.\n");
        return;
    }
    heap->data[0] = heap->data[heap->size - 1];

```

```

    heap->size--;
    heapify_down(heap, 0);
    printf("CEO has read email and dealt with it\n");
}
// Display count of unread emails (COUNT command implementation)
void get_count(MaxHeap *heap) {
    printf("Current unread emails: %d\n", heap->size);
}
// Free allocated memory
void free_heap(MaxHeap *heap) {
    free(heap->data);
    free(heap);
}
int main() {
    MaxHeap *inbox = create_heap(10);
    // Example testing commands matching the specifications
    insert_email(inbox, "Peer", "Project Update", "10-15-2026");
    insert_email(inbox, "Boss", "Q3 Budget Review", "10-18-2026");
    insert_email(inbox, "Boss", "Urgent Meeting", "10-20-2026"); // Newer boss email should take precedence
    printf("--- COUNT ---\n");
    get_count(inbox);
    printf("\n--- NEXT ---\n");
    peek_next(inbox);
    printf("\n--- READ ---\n");
    read_email(inbox);
    printf("\n--- COUNT AFTER READ ---\n");
    get_count(inbox);
    printf("\n--- NEXT AFTER READ ---\n");
    peek_next(inbox);
    free_heap(inbox);
    return 0;
}

```

e)

- **Correctness:** Both programs correctly use a MaxHeap, sender priorities, and date-based tie-breaking. However, Claude's program is more suitable for the assignment because it reads input from a file and supports all required commands (EMAIL, NEXT, READ, and COUNT). Gemini uses hard-coded test data in main, so it would require a lot of changes to work for the grader. Claude also uses safer string handling and checks memory allocation compared to Gemini because it uses functions such as strcpy without checking the size of the destination arrays.
- **Execution Time:** Both programs have the same overall time complexity. Adding an email to the heap takes $O(\log n)$ time because the new email may need to move up the heap.

Removing the highest-priority email with READ also takes $O(\log n)$ time because the remaining heap may need to be rearranged, while NEXT and COUNT take $O(1)$.

Therefore, neither program has a significant time-complexity advantage over the other.

- **Space Complexity:** Both programs use $O(n)$ space to store emails. Claude's iterative heap operations use $O(1)$ auxiliary space, while Gemini's recursive heap operations use $O(\log n)$ stack space. Their overall space complexity remains $O(n)$ because the heap stores all the emails.
- **Maintainability:** Claude's code is easier to maintain because it separates heap operations, input processing, and command handling into different functions. This makes it easier to understand the program and modify. Gemini also uses separate functions, but its hard-coded test data makes it less adaptable. Claude's safer memory and string handling also improve maintainability and make it easier to make changes.

f) I am selecting Claude's code because it is closer to the assignment requirements. It reads commands from input and supports EMAIL, NEXT, READ, and COUNT, while Gemini's code uses hard-coded test data. Claude's code also has safer string handling, checks memory allocation, and uses iterative heap operations. Therefore, Claude requires fewer changes to become a complete solution for the assignment.

g) **Correctness:** I modified Claude's code to match the required output format and ensured that NEXT and READ work correctly when the heap is empty. I also made sure the program can process any number of commands, including repeated NEXT and READ commands.

Execution Time: No major changes were made because Claude's heap operations were already efficient. EMAIL and READ remain $O(\log n)$, while NEXT and COUNT remain $O(1)$.

Therefore, the program maintains efficient execution time as the number of emails increases.

Space Complexity: No major changes were made. The program still uses $O(n)$ space for the heap. I kept Claude's iterative heap operations instead of changing them to recursive functions.

The iterative heap operations use $O(1)$ additional space and don't use $O(\log n)$. Therefore, the program uses the same space complexity and uses minimal additional memory usage.

Maintainability: I improved the code by adding clearer prologue and inline comments, organizing the code into separate functions, and making the command processing easier to follow. These changes make the program easier to understand, test, and modify.